

DIAGNOSING JEPA WORLD MODELS WITH ACTION-CONDITIONED PREDICTIVE CONSISTENCY

Anonymous authors

Paper under double-blind review

ABSTRACT

Joint-embedding predictive architectures (JEPAs) learn world models that predict in a compact latent space rather than in pixels, reducing the pressure to model nuisance appearance. Yet this provides no guarantee against visual perturbations: they can still alter the encoded representation and affect subsequent action-conditioned predictions. Bisimulation captures this requirement precisely: two observations should be treated as the same state only when their action-conditioned consequences agree. Guided by this criterion, we introduce Action-Conditioned Predictive Consistency (ACPC), a diagnostic that measures how far a clean history and a visually perturbed view of it diverge after being rolled forward under the same action sequence. We prove that this divergence bounds the perturbation-induced change in multi-step prediction error and planner cost. Building on pairwise ACPC, we define two complementary measures: the Invariance Radius (IR) summarizes clean-perturbed rollout spread, while the Separation Rate (SR) checks whether different states remain distinguishable after rollout. Experiments on four visual control tasks show that pairwise ACPC predicts perturbation-induced prediction and cost changes. On LeWM, the IR-SR screen transfers across tasks, and both IR and SR remain informative under blur and resize. PLDM exhibits similar diagnostic trends under a different architecture.

1 INTRODUCTION

World models support control by predicting the outcomes of candidate actions Hafner et al. (2025); Hansen et al. (2024). Joint-embedding predictive architectures (JEPAs) predict target representations rather than reconstructing pixels, avoiding an explicit requirement to reproduce nuisance visual details LeCun (2022); Assran et al. (2023); Bardes et al. (2024); Van Assel et al. (2025); Littwin et al. (2024). Yet this design alone does not determine which visual differences matter for control. Such representations should be insensitive to task-irrelevant visual perturbations while preserving distinctions between situations that require different actions. This requirement echoes the intuition behind bisimulation, which defines state equivalence through action-conditioned consequences rather than visual appearance Gelada et al. (2019); Zhang et al. (2021). However, encoder distances alone do not show how a perturbation propagates through prediction. Over a multi-step rollout, the predictor may amplify or contract the initial representation difference. Encoder-level and single-transition diagnostics do not directly measure this rollout-level effect. We therefore ask: when a clean history and its perturbed view are rolled forward under the same action sequence, how far apart are their predicted trajectories?

To measure this rollout-level effect, we introduce Action-Conditioned Predictive Consistency (ACPC). ACPC rolls a clean history and its perturbed view forward under the same action sequence and measures the distance between their predicted trajectories. Figure 1 provides a conceptual overview of how this pairwise distance is summarized by IR and complemented by SR. Low ACPC alone, however, does not rule out representational collapse: a constant representation would make every paired rollout identical. We therefore summarize ACPC across histories with the *Invariance Radius* (IR), where lower values indicate less sensitivity to visual perturbations. The *Separation Rate* (SR) checks whether different states remain distinguishable after rollout, with higher values indicating better separation.

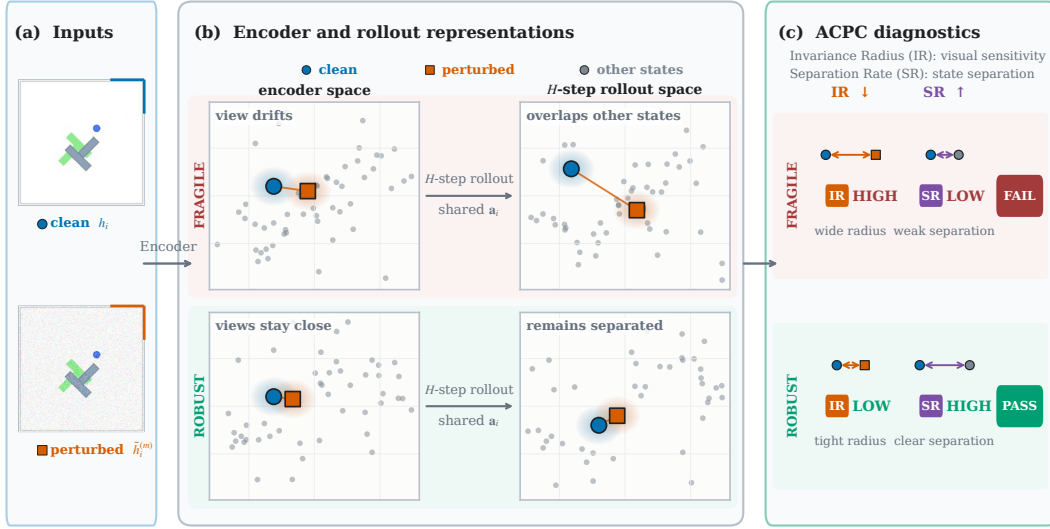


Figure 1: **Conceptual overview of ACPC, IR, and SR.** (a) A logged clean history and a visually perturbed view of it form the paired inputs. (b) A frozen world model encodes both views and rolls them forward for H steps under the same recorded action sequence. ACPC measures the distance between the two predicted trajectories. (c) The Invariance Radius (IR) summarizes perturbation-induced rollout spread across histories, while the Separation Rate (SR) measures how often selected different-state rollouts remain separated beyond this spread.

We make three contributions. First, we introduce **Action-Conditioned Predictive Consistency (ACPC)**, which compares clean and perturbed rollout predictions under shared actions; IR summarizes sensitivity and SR checks state separation (Section 3). Second, we prove samplewise **bounds on prediction-error and planning-cost changes** over multi-step rollouts with identical candidate action sequences across views (Propositions 1 and 2). These bounds require no distributional or smoothness assumptions. Third, we develop **ACPC-based checkpoint screening** using IR and SR and evaluate all three diagnostics on four tasks, three visual perturbations, and two architectures. On LeWM Maes et al. (2026), ACPC is informative about prediction-error change and extra predicted cost of perturbation-induced plan changes in the cross-entropy method (CEM) Kroese et al. (2006). Across checkpoints, IR and SR provide non-equivalent views of perturbation sensitivity and tested state separation; their continuous values covary with retained control performance (Section 4).

2 RELATED WORK

World Models. World models predict how an environment evolves under actions and use those predictions for control. DreamerV3 Hafner et al. (2025) learns from imagined trajectories, while TD-MPC2 Hansen et al. (2024) plans with learned latent dynamics. Joint-embedding predictive architectures (JEPAs) learn representations by predicting targets in representation space rather than reconstructing pixels LeCun (2022); Assran et al. (2023); Bardes et al. (2024); Assran et al. (2025). LeWM Maes et al. (2026) presents a simple approach to stable end-to-end training of a visual encoder and an action-conditioned latent predictor from raw pixels. Its objective pairs next-step latent prediction with LeJEPa’s SIGReg Balestriero & LeCun (2025), which encourages a Gaussian latent distribution to prevent collapse. PLDM Sobal et al. (2025) learns representations and action-conditioned latent dynamics from reward-free offline trajectories, then uses the learned dynamics for goal-conditioned planning. PLDM uses a VICReg-inspired anti-collapse objective Bardes et al. (2022), whose variance and covariance terms maintain feature variation and decorrelate features, respectively. Separately, VISReg Wu et al. (2026) retains variance regularization but replaces VICReg’s covariance regularization with Sliced-Wasserstein-based sketching. Related work studies collapse in self-predictive learning Tang et al. (2023) and the role of action conditioning Khetarpal et al. (2025). Theoretical analyses ask which features latent prediction retains Van Assel et al. (2025);

Littwin et al. (2024). seq-JEPA Ghaemi et al. (2025) further studies invariance and equivariance under known view transformations.

Robustness to Visual Perturbations. Training-time augmentation improves visual control in DrQ Yarats et al. (2021) and DrQ-v2 Yarats et al. (2022), while SODA Hansen & Wang (2021) uses augmentation in an auxiliary representation-learning objective. ReOI Chen et al. (2026) instead modifies observations at test time to reduce distractor sensitivity. Other methods change world-model training. DreamerPro Deng et al. (2022) uses prototype prediction, Denoised MDPs Wang et al. (2022) retain information that is controllable and reward-relevant, and another approach combines temporal masking with bisimulation Sun et al. (2024). MWM Yan et al. (2026) enforces action-conditioned rollout consistency during training. These methods reflect a broader principle: robustness should remove irrelevant visual variation without discarding distinctions needed for control. Bisimulation formalizes behavioral similarity through rewards and action-conditioned transitions Gelada et al. (2019); Zhang et al. (2021); Shimizu & Tomizuka (2025), including a reward-free formulation based only on transitions Toso et al. (2026).

Diagnostics for World Models. Model accuracy and control performance do not always move together: one-step likelihood can be a poor guide to downstream performance Lambert et al. (2020). This mismatch has motivated methods that connect model learning and evaluation to downstream decisions Wei et al. (2024). World-model evaluation should therefore reflect the intended use of the model, including long rollouts, planning, and policy evaluation Yu et al. (2026). One group of diagnostics examines action information in learned transitions and the validity of long rollouts. ATM Chen (2026) compares action information in encoded and predicted transitions. Frozen inverse-dynamics probes test whether latents retain action information under visual corruptions Yeom et al. (2026), while other diagnostics expose kinematic failures in long rollouts Schäfer et al. (2026). A second group measures prediction errors or tests world-model agents with visual attacks. Some methods compare predicted multi-step latent transitions with those produced by the environment Vakalis (2026), or compare predicted and true plan costs You et al. (2026). WMAAttack Guo et al. (2026) searches for visual attacks on closed-loop world-model agents, while ARB4WM Zhang et al. (2026) targets their policy, value, and latent dynamics. Finally, CARRL Lütjens et al. (2020) certifies Q-based action choices under bounded observation uncertainty, while CROP Wu et al. (2022) certifies per-state actions and finite-horizon reward bounds through smoothing.

3 ACTION-CONDITIONED PREDICTIVE CONSISTENCY AS A DIAGNOSTIC

A visual perturbation can change the trajectory predicted by a world model. ACPC measures this change by rolling a clean history and its perturbed version forward under the same action sequence and comparing the two trajectories. We first define this pairwise measurement. We then show that it bounds perturbation-induced changes in prediction error and candidate cost, which leads to conditions for preserving a planner’s selected candidate. To summarize ACPC across a checkpoint, IR captures clean–perturbed pairs with high sensitivity, while SR checks that selected differently labeled histories remain separated and can flag collapse over those tested distinctions.

3.1 PAIRWISE ACPC

Given a clean history and its perturbed version, ACPC compares the rollouts predicted from them under identical actions. Let h denote the clean history and let $\tilde{h}$ be the result of applying the evaluated visual perturbation to h . The frozen encoder maps them to $z = E_\theta(h)$ and $\tilde{z} = E_\theta(\tilde{h})$. Both representations are then rolled forward under the action sequence $\mathbf{a} = (a_0, \dots, a_{H-1})$. Let F_θ denote the frozen action-conditioned predictor. After the first k actions, the two predicted representations are

$$\hat{z}_k = F_\theta^k(z, \mathbf{a}_{0:k-1}), \quad \hat{\tilde{z}}_k = F_\theta^k(\tilde{z}, \mathbf{a}_{0:k-1}). \quad (1)$$

Here H is the number of autoregressive future steps. ACPC compares predictions in the projected latent space used to evaluate planner costs. Let Π map each predicted representation into this space. To combine all H predicted steps into one trajectory-level distance, we assign nonnegative weights α_k , with $\sum_{k=1}^H \alpha_k = 1$, and define the weighted rollout vector

$$\bar{G}_\mathbf{a}(z) = [\sqrt{\alpha_1}\Pi(\hat{z}_1)^\top \quad \dots \quad \sqrt{\alpha_H}\Pi(\hat{z}_H)^\top]^\top. \quad (2)$$

Action-Conditioned Predictive Consistency (ACPC) is the distance between the two weighted predicted rollouts:

$$\text{ACPC}_H(h, \tilde{h}, \mathbf{a}) = \left\| \bar{G}_{\mathbf{a}}(E_{\theta}(h)) - \bar{G}_{\mathbf{a}}(E_{\theta}(\tilde{h})) \right\|_2 = \left(\sum_{k=1}^H \alpha_k \left\| \Pi(\hat{z}_k) - \Pi(\hat{\tilde{z}}_k) \right\|_2^2 \right)^{1/2}, \quad (3)$$

Low ACPC means that the two predicted trajectories remain close; high ACPC means that they separate. Encoder shift $\|z - \tilde{z}\|_2$ measures the difference before prediction, whereas ACPC measures it after rollout. Using the same action sequence removes action differences from the comparison. ACPC measures rollout consistency, not prediction accuracy.

3.2 PREDICTION-ERROR BOUNDS

ACPC itself does not require the observed future. To connect it to prediction error, let $Y_{\mathbf{a}}^H$ denote the observed future under the same action sequence, represented in the same weighted space as the predicted rollouts. Since the perturbed history is created by applying a visual perturbation to the clean history, both predicted rollouts are compared with the same observed future.

Proposition 1 (Prediction-error change bound). *For $e_h = \|\bar{G}_{\mathbf{a}}(E_{\theta}(h)) - Y_{\mathbf{a}}^H\|_2$ and the analogous $e_{\tilde{h}}$, every paired sample satisfies*

$$|e_{\tilde{h}} - e_h| \leq \text{ACPC}_H(h, \tilde{h}, \mathbf{a}). \quad (4)$$

This result follows directly from the reverse triangle inequality; the proof is given in Appendix A. The bound concerns the change in error, not absolute prediction accuracy. Both rollouts can be inaccurate even when ACPC is small. In the experiment, the observed future is used to compute the error change, not ACPC itself (Section 4.4).

3.3 SELECTION STABILITY FROM PLANNING-COST BOUNDS

A planner ranks candidate action sequences by their predicted costs. Changing the input can change each candidate’s predicted endpoint and therefore its cost. If these cost changes are too small to close the gaps between candidates, the selected candidate remains unchanged.

For each candidate action sequence $\mathbf{a}^j$, we roll both inputs forward under that sequence. Let

$$x_j = \Pi(F_{\theta}^H(E_{\theta}(h), \mathbf{a}^j)), \quad \tilde{x}_j = \Pi(F_{\theta}^H(E_{\theta}(\tilde{h}), \mathbf{a}^j))$$

be the final clean and perturbed predicted representations. Their displacement $r_j = \|x_j - \tilde{x}_j\|_2$ is one component of candidate-specific ACPC. Whenever $\alpha_H > 0$,

$$r_j \leq \frac{\text{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)}{\sqrt{\alpha_H}}.$$

Proposition 2 (Planning-cost bounds). *Suppose the planner scores candidate j by its squared distance to a fixed goal embedding g :*

$$C_j = \|x_j - g\|_2^2, \quad \tilde{C}_j = \|\tilde{x}_j - g\|_2^2.$$

Define the candidate-specific cost-change bound

$$b_j = r_j(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2). \quad (5)$$

Then $|\tilde{C}_j - C_j| \leq b_j$.

Let w and $\tilde{w}$ be the winners under the clean and perturbed inputs. If the winner changes, the perturbed winner’s excess cost under the clean prediction satisfies

$$0 \leq C_{\tilde{w}} - C_w \leq b_{\tilde{w}} + b_w. \quad (6)$$

The bound b_j is computed from the evaluated endpoints and requires no global Lipschitz constant. If the clean winner leads every competitor by more than $b_w + b_j$, the perturbation cannot reverse their order. The same argument preserves the top- k elite set when every clean elite–non-elite gap exceeds the corresponding pair of bounds. If paired CEM runs start from the same proposal and preserve the elite set in every round, they generate the same later candidates and return the same action (Corollary 1). Once an elite set differs, this guarantee no longer applies. This result concerns the planner’s model-based choice, not its return in the environment.

3.4 CHECKPOINT-LEVEL IR AND SR

Pairwise ACPC measures one clean-perturbed pair under one action sequence. Evaluating a checkpoint requires summarizing this measurement across many histories. The *Invariance Radius* (IR) measures how sensitive these paired rollouts are to the visual perturbation; lower IR is better. Low IR alone is not enough because a collapsed representation would make every rollout look similar. The *Separation Rate* (SR) therefore checks whether sampled histories with different state-coordinate labels remain separated after rollout; higher SR is better.

To compute IR, we select n logged history windows as anchors. Anchor i provides a clean history h_i and its recorded action sequence $\mathbf{a}_i = (a_{i,0}, \dots, a_{i,H-1})$. We apply the visual perturbation M times to obtain $\tilde{h}_i^{(1)}, \dots, \tilde{h}_i^{(M)}$, and compute ACPC for each clean-perturbed pair under the same recorded actions. Raw latent distances can have different scales across histories. We therefore divide each ACPC value by s_i , the median projected one-step clean motion over the next H observed transitions. With a small numerical stabilizer $\varepsilon_{\text{norm}}$, define

$$R_i^{(m)} = \frac{\text{ACPC}_H(h_i, \tilde{h}_i^{(m)}, \mathbf{a}_i)}{s_i + \varepsilon_{\text{norm}}}, \quad \bar{R}_i = \frac{1}{M} \sum_{m=1}^M R_i^{(m)}. \quad (7)$$

Let Q_q denote the empirical q -quantile across anchors. The raw IR is

$$\text{IR}_q^{\text{raw}}(\theta) = Q_q(\{\bar{R}_i\}_{i=1}^n). \quad (8)$$

For each eligible anchor, the protocol selects a nearby logged history with a different state-coordinate label. Both histories are rolled forward under the anchor’s recorded actions, so their separation is not caused by different action sequences. Their trajectory distance is normalized by the same clean-motion scale used for IR and is denoted by D_i^{diff} . Let $\mathcal{I}$ be the set of anchors for which such a comparison is available. SR is the fraction whose different-state distance exceeds raw IR by a margin δ :

$$\text{SR}_{q,\delta}(\theta) = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbf{1}[D_i^{\text{diff}} > \text{IR}_q^{\text{raw}}(\theta) + \delta], \quad (9)$$

We use $q = 0.90$ and $\delta = 0.10$. A collapsed representation makes IR small, but it also makes every D_i^{diff} zero and therefore fails SR.

3.5 CHECKPOINT SCREENING

SR uses raw IR because it compares distances within the same checkpoint. To compare a trained checkpoint with its unaugmented reference, we use relative IR. Let θ_0 denote the unaugmented checkpoint from the same task, training run, and model family. We define

$$\text{IR}_q^{\text{rel}}(\theta; \theta_0) = \frac{\text{IR}_q^{\text{raw}}(\theta)}{\text{IR}_q^{\text{raw}}(\theta_0)}. \quad (10)$$

Relative IR equals 1 for the reference checkpoint.

A checkpoint passes the screen only when relative IR is at or below its threshold and SR is at or above its threshold. This is a Boolean screen, not a continuous ranking. We choose its thresholds using planning success on a set of source tasks and apply them unchanged to the remaining tasks. After calibration, planning success is no longer an input to the screen. These checkpoint-level findings are empirical; they do not follow from the pairwise bounds.

The experiments evaluate the method at two levels. Pair-level experiments test whether ACPC reflects changes in prediction error and planner selection (Section 4.4 and Section 4.5). Checkpoint-level experiments first select an IR-SR screen on some LeWM tasks and test it on the remaining tasks under Gaussian noise.

4 EXPERIMENTS

4.1 EVALUATION PROTOCOL

Models, Tasks, and Evaluation. We use four control tasks: TwoRoom navigation, PushT planar manipulation, Reacher arm control, and OGBench-Cube 3D manipulation. For planning evaluation,

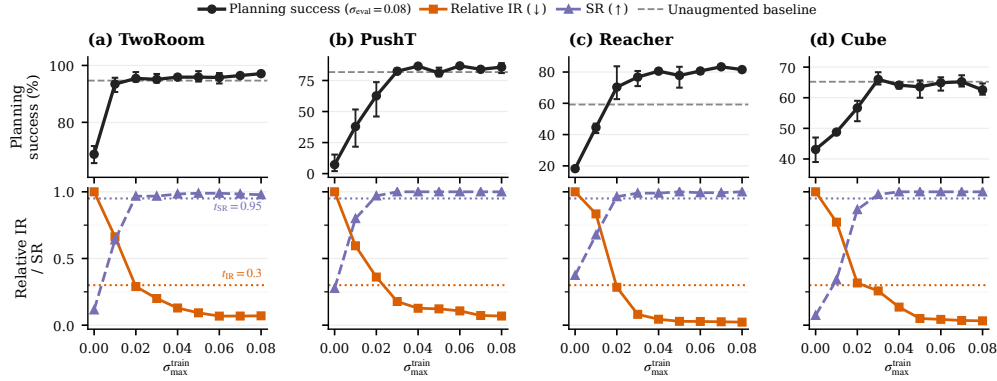


Figure 2: Checkpoint-level planning performance and diagnostics across the complete Gaussian-noise training sweep. Points are across-run means; success-rate error bars span the three training runs, and success-rate axes are scaled per task. Lower IR and higher SR are favorable.

CEM uses the frozen world model to optimize five-step action sequences. We report the percentage of episodes that reach the task goal. The main experiments add Gaussian noise with $\sigma = 0.08$ to the history observations while keeping the goal image clean. For each task and model family, the Gaussian-noise sweep contains nine training conditions: one without noise augmentation and eight with full-sequence Gaussian-noise augmentation at $\sigma_{\max} \in \{0.01, \dots, 0.08\}$.

Diagnostic Protocol. IR uses 100 logged anchors, five Gaussian-noise draws at the evaluation severity $\sigma = 0.08$ (blur and resize diagnostics instead apply the corresponding shift), eight predicted steps, uniform horizon weights, within-anchor draw averaging, and a q90 summary across anchors. To compute SR, the protocol selects, for each anchor, a nearby history with a different endpoint-state label and checks whether the two rollouts remain farther apart than raw IR plus 0.10.

Broad-severity protocol. We also conduct a separate LeWM experiment with $\sigma_{\max} \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.7, 1.0, 1.5\}$ and $\sigma_{\text{eval}} \in \{0.08, 0.16, 0.32, 0.64\}$. During training, every frame is perturbed and its noise standard deviation is sampled independently as $\sigma_{\text{train}} \sim \text{Uniform}(0, \sigma_{\max})$. The eight-step diagnostic, state-pair catalog, normalization, and thresholds remain unchanged. All reported task-condition pairs use three independent training runs. Each checkpoint is evaluated with three evaluation seeds and 100 episodes per seed; the training run is the unit of replication.

Threshold Protocol. A checkpoint meets the success-rate criterion if it recovers at least 80% of the improvement from the unaugmented checkpoint to the highest success in the fixed nine-checkpoint sweep, while losing no more than five percentage points on clean observations.

4.2 IR AND SR ACROSS CHECKPOINT RECOVERY

Gaussian-noise augmentation recovers performance over a range of training levels whose location differs by task (Figure 2). On Reacher, augmentation also raises clean success from 59.2% to 76–82%. Its recovery under noisy evaluation therefore cannot be attributed to robustness alone; the checkpoints also improve on clean observations.

Every augmentation level that meets the success-rate criterion has lower IR and higher SR than its unaugmented reference, although neither measure is monotone at every level. Across the 108 LeWM checkpoint rows, 77 pass the reported IR threshold $t_{\text{IR}} = 0.3$, and all 77 also pass $t_{\text{SR}} = 0.95$.

4.3 IR AND SR UNDER BROADER EVALUATION SEVERITIES

We test whether this redundancy persists in the separate broad-severity experiment.

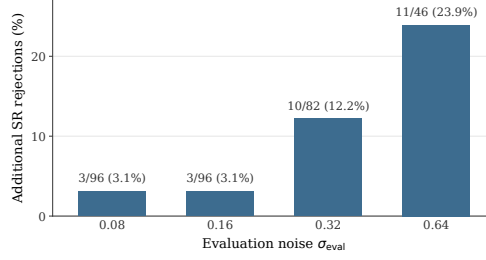


Figure 3: **SR provides a distinct fixed-threshold constraint beyond relative IR.** Among checkpoints satisfying $\text{IR}^{\text{rel}} \leq 0.3$, each bar shows the percentage rejected by $\text{SR} < 0.95$; annotations give the rejection count and the number passing IR.

SR rejects 3/96 IR-passing rows at $\sigma_{\text{eval}} = 0.08$ and 3/96 at 0.16, increasing to 10/82 and 11/46 at the two harder severities. The conditions also disagree in the other direction: at both $\sigma_{\text{eval}} = 0.32$ and 0.64, 5 and 5 rows, respectively, pass SR but fail relative IR.

We next relate the continuous diagnostics to same-checkpoint retention, defined as success under noise minus clean success. After accounting for the simple exposure ratio $-\sigma_{\text{eval}}/\sigma_{\text{max}}$, the correlations remain positive when tasks are weighted equally: 0.458 for IR and 0.532 for SR. The adjusted IR association is near zero on PushT (0.070), so the result is not uniform across tasks. These associations do not establish better checkpoint selection or replace evaluation of nominal model quality.

4.4 ACPC AND PREDICTION-ERROR CHANGE

We test whether ACPC helps predict how much a visual perturbation changes multi-step prediction error, $d = |e_{\tilde{h}} - e_h|$. We evaluate at the protocol horizon $H = 8$, the longest horizon with a complete logged common future for every anchor. The analysis uses the unaugmented checkpoint of each task and training run. Trajectories are partitioned into 16 disjoint groups. Each history pair contributes rows at Gaussian-noise standard deviations $\sigma \in \{0.02, 0.05, 0.08\}$ with two perturbation draws; zero-severity probes serve only as identity checks. A ridge model is fitted on 15 groups and evaluated on the remaining group, rotating through all 16, and all rows from one trajectory group stay together.

Every ridge model contains the probe severity, encoder-history distance, and one-step ACPC under the recorded action. Base+Control₈ adds the strongest destroyed-action control: eight-step ACPC recomputed with zeroed, swapped, or temporally shuffled actions. Base+ACPC₈ instead adds eight-step ACPC under the recorded actions. For each task-run cell, we report the control with the lowest test MAE among the three, making this a conservative comparison.

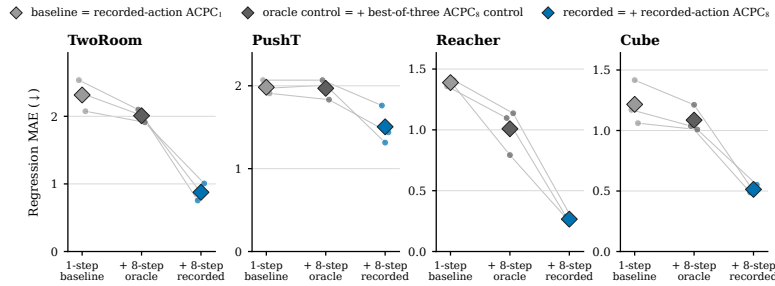


Figure 4: Cross-validated MAE for predicting the error drift d on the unaugmented checkpoints; lower is better.

Base+ACPC₈ attains the lowest cross-validated MAE in all 12 task-run cells (Figure 4). It reduces held-out MAE by $55.9 \pm 4.7\%$ relative to Base and by $51.3 \pm 3.5\%$ relative to Base+Control₈ (mean $\pm$ sample standard deviation across the three runs). These results concern prediction of the error drift; they do not compare absolute prediction accuracy across rollout horizons.

4.5 ACPC AND THE COST OF CEM PLAN CHANGES

A visual perturbation may cause CEM to select a different plan. We ask whether ACPC reflects how costly that change appears to the clean model. Let $\mathbf{a}$ and $\tilde{\mathbf{a}}$ be the final plans selected from the clean and perturbed histories, respectively. We measure $(C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a}))_+$: the extra clean-history model cost incurred by selecting the perturbed-history plan. We refer to this single-decision quantity as *CEM selection regret*. It uses the model’s squared latent goal cost; it is neither cumulative regret nor simulator return.

The fixed-pool bound motivates this comparison. However, adaptive CEM can generate different later pools once the two elite sets diverge. We therefore evaluate full paired CEM runs rather than treating the fixed-pool condition as a run-level certificate. The paired CEM branches follow Appendix I: same initial proposal, common random numbers, each branch updating from its own elites. For each candidate pool, we compute one- and five-step ACPC for every action sequence and use the 90th percentile as the pool summary. The baseline uses perturbation severity, training condition, candidate-level one-step ACPC q90, and the clean gap between the best and second-best candidates. The expanded model adds candidate-level five-step ACPC q90. Within each training run, both regressions are fitted on three tasks and tested on the remaining task, rotating which task is excluded from fitting. We evaluate MAE on $\log(1 + \text{selection regret})$; Appendix I gives the CEM budget and sample counts.

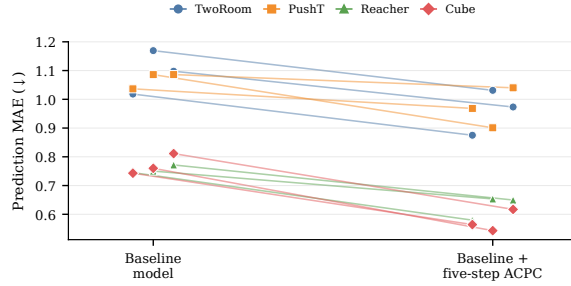


Figure 5: Adding planner-horizon ACPC improves prediction of the extra model cost associated with changes in the plan selected by adaptive CEM.

Adding planner-horizon ACPC lowers cross-task test MAE by 15.2% (a 2.0 percentage-point sample standard deviation across training runs). The error decreases in all 12 task–run test cases (Figure 5). On fixed first-round CEM candidate pools, the top-1 sufficient condition covers 34.8–69.5% of histories for augmented checkpoints across probe severities, versus 0.0–2.3% for unaugmented checkpoints. Appendix I reports the full fixed-pool analysis; it does not certify later adaptive rounds.

4.6 CHECKPOINT SCREENING ACROSS TASKS

We test whether thresholds chosen on some tasks identify checkpoint recovery on the remaining tasks. For the two- and three-source settings, the first accepted checkpoint is within 0.5 grid steps of recovery on average and never differs by more than one step (balanced accuracy 0.900, precision/recall 0.913/0.953). Full results for all 14 choices of source tasks appear in Table 9 in Appendix F.

4.7 DIAGNOSTIC BEHAVIOR ON PLDM

We apply the same ACPC, IR, and SR definitions to PLDM, which uses a different architecture and training recipe.

Table 1: **PLDM diagnostics across the augmented training grid.** Values are three-run means; brackets give their range across all eight augmented levels.

Task	Planning success (%)	Relative IR	SR
	unaug. → augmented range	augmented range	unaug. → augmented range
TwoRoom	$67.1 \pm 8.0 \rightarrow [95.7, 97.7]$	$[0.071, 0.440]$	$0.082 \rightarrow [0.967, 1.000]$
PushT	$10.2 \pm 5.9 \rightarrow [20.9, 70.3]$	$[0.070, 0.866]$	$0.228 \rightarrow [0.439, 1.000]$
Reacher	$53.0 \pm 29.7 \rightarrow [63.0, 82.2]$	$[0.166, 0.893]$	$0.650 \rightarrow [0.887, 1.000]$
Cube	$45.9 \pm 1.7 \rightarrow [50.6, 53.9]$	$[0.043, 0.742]$	$0.170 \rightarrow [0.637, 1.000]$

A similar descriptive low-IR, high-SR pattern therefore appears in the evaluated PLDM sweeps. We do not transfer thresholds between model families, so this result does not establish a shared calibrated decision boundary.

4.8 CHECKPOINT COMPARISON UNDER BLUR AND RESIZE

To test whether the diagnostics remain informative beyond Gaussian noise, we evaluate one blur severity ($k = 15$) and one resize severity (scale 0.25). For each task, training run, and visual shift, we compare the unaugmented checkpoint with the checkpoint trained at $\sigma_{\max} = 0.08$, giving $4 \times 3 \times 2 = 24$ pairs. We recompute IR and SR under the evaluated shift and report the components separately. Because relative IR equals one for the reference checkpoint, we define IR improvement as $1 - \text{IR}_q^{\text{rel}}$ at the augmented endpoint. SR change is the endpoint SR minus reference SR. Positive values favor the augmented checkpoint for both components. The thresholds selected under Gaussian noise remain fixed; no blur- or resize-specific calibration is performed.

Treating a positive component change as a positive prediction, IR improvement and SR change each match the prespecified binary behavioral label in 22 of the 24 comparisons. Their pooled Spearman correlations with planning-success change are 0.854 and 0.913, respectively; after omitting one task at a time, the ranges are $[0.720, 0.911]$ for IR and $[0.846, 0.901]$ for SR.

Only 6 of the 24 augmented endpoints pass the fixed IR threshold, and all six also pass SR. Thus, SR changes no pass/fail decision beyond IR in this experiment. We therefore treat the component associations as descriptive rather than evidence that SR improves the Boolean screen.

5 DISCUSSION AND LIMITATIONS

What is supported. For individual clean-perturbed pairs, multi-step ACPC captures changes in prediction error more accurately than encoder distance, one-step ACPC, or controls that remove the recorded action sequence. At the planning horizon, ACPC also helps predict the extra model cost incurred when a perturbation changes the plan selected by CEM, beyond one-step ACPC or the clean CEM margin.

Scope and limitations. ACPC diagnoses how visual perturbations affect predicted rollouts and planning costs; it does not modify CEM. Our planner experiment therefore measures whether ACPC reflects the extra predicted cost of a perturbation-induced plan change, rather than whether ACPC-guided planning improves task success. The adaptive-CEM guarantee applies only while its bound verifies that both runs select the same elite candidates. The full IR-SR screen requires an unaugmented reference checkpoint. Unlike pairwise ACPC, it also uses observed future frames to normalize IR and dataset state labels to construct SR pairs. The state-pair catalog limits which distinctions SR can assess, and nominal clean quality remains a separate requirement. The broad Gaussian experiment covers several evaluation severities, but blur and resize remain limited to one severity each, and we do not test natural changes in background, lighting, camera pose, or occlusion. The chosen IR threshold is also at the edge of the tested range, so larger values remain to be evaluated.

6 CONCLUSION

ACPC measures how much a clean observation and its visually perturbed counterpart diverge after both are rolled forward under the same action sequence. We show that this divergence bounds changes in multi-step prediction error and predicted planning costs. IR and SR serve complementary diagnostic roles: IR measures clean-perturbed rollout spread, while SR checks whether different states remain distinguishable after rollout. On LeWM, the IR-SR checkpoint screen transfers across tasks, and both diagnostics remain informative under blur and resize. Across the evaluated checkpoints, LeWM and PLDM show similar trends in both IR and SR.

AI USE STATEMENT

Generative AI tools were used to assist with code generation and debugging, language editing, and auxiliary checks of experimental results. The authors specified the research questions, methods, and experimental protocols; independently verified all AI-assisted code, proofs, numerical results, and manuscript claims; and take full responsibility for the submission.

REPRODUCIBILITY STATEMENT

The appendix specifies tasks, perturbations, checkpoints, seeds, sampling, normalization, thresholds, and aggregation. The anonymous supplementary material includes the code and machine-readable results needed to reproduce the reported figures and tables.

REFERENCES

- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15619–15629, 2023. doi: 10.1109/CVPR52729.2023.01499.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. arXiv preprint arXiv:2506.09985, 2025. URL <https://arxiv.org/abs/2506.09985>.
- Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. arXiv preprint arXiv:2511.08544, 2025. URL <https://arxiv.org/abs/2511.08544>.
- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022. URL <https://openreview.net/forum?id=xm6YD62D1Ub>.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=QaCCuDfBk2>.
- Jiaheng Chen. ATM: Action-consistency transfer matrix for diagnosing and improving latent world models. arXiv preprint arXiv:2606.09028, 2026. URL <https://arxiv.org/abs/2606.09028>.
- Yuxin Chen, Jianglan Wei, Chenfeng Xu, Boyi Li, Masayoshi Tomizuka, Andrea Bajcsy, and Ran Tian. Reimagination with test-time observation interventions: Distractor-robust world model predictions for visual model predictive control. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026. URL <https://re-oi.github.io/>.
- Fei Deng, Ingook Jang, and Sungjin Ahn. DreamerPro: Reconstruction-free model-based reinforcement learning with prototypical representations. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 4956–4975. PMLR, 2022. URL <https://proceedings.mlr.press/v162/deng22a.html>.
- Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2170–2179. PMLR, 2019. URL <https://proceedings.mlr.press/v97/gelada19a.html>.

- Hafez Ghaemi, Eilif B. Muller, and Shahab Bakhtiari. seq-JEPA: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pp. 32943–32973, 2025. doi: 10.52202/085713-1104.
- Zhixiang Guo, Siyuan Liang, Shi Fu, Cheng Guo, András Balogh, Márk Jelasity, and Dacheng Tao. WMAttack: Automated attack search for adversarial evaluation of world-model agents. arXiv preprint arXiv:2605.23220, 2026. URL <https://arxiv.org/abs/2605.23220>.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025. doi: 10.1038/s41586-025-08744-2.
- Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 13611–13617, 2021. doi: 10.1109/ICRA48506.2021.9561103.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=Oxh5CstDJU>.
- Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 18(3):1059–1076, 1989. doi: 10.1080/03610918908812806.
- Khimya Khetarpal, Zhaohan Daniel Guo, Bernardo Avila Pires, Yunhao Tang, Clare Lyle, Mark Rowland, Nicolas Heess, Diana L. Borsa, Arthur Guez, and Will Dabney. A unifying framework for action-conditional self-predictive reinforcement learning. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pp. 181–189. PMLR, 2025. URL <https://proceedings.mlr.press/v258/khetarpal25a.html>.
- Dirk P. Kroese, Sergey Porotsky, and Reuven Y. Rubinstein. The cross-entropy method for continuous multi-extremal optimization. *Methodology and Computing in Applied Probability*, 8(3):383–407, 2006. doi: 10.1007/s11009-006-9753-0.
- Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, volume 120 of *Proceedings of Machine Learning Research*, pp. 761–770. PMLR, 2020. URL <https://proceedings.mlr.press/v120/lambert20a.html>.
- Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022. URL <https://openreview.net/forum?id=BZ5alr-kVsf>. Version 0.9.2.
- Etai Littwin, Omid Saremi, Madhu Advani, Vimal Thilak, Preetum Nakkiran, Chen Huang, and Joshua Susskind. How JEPA avoids noisy features: The implicit bias of deep linear self distillation networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pp. 91300–91336, 2024. doi: 10.52202/079017-2897.
- Björn Lütjens, Michael Everett, and Jonathan P. How. Certified adversarial robustness for deep reinforcement learning. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pp. 1328–1337. PMLR, 2020. URL <https://proceedings.mlr.press/v100/lutjens20a.html>.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. arXiv preprint arXiv:2603.19312, 2026. URL <https://arxiv.org/abs/2603.19312>.
- Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. In Lise Getoor and Tobias Scheffer (eds.), *Proceedings of the 28th International Conference on Machine Learning*, pp. 833–840. Omnipress, 2011. URL https://icml.cc/2011/papers/455_icmlpaper.pdf.

- Finn Rasmus Schäfer, Korbinian Moller, Yuan Gao, Christian Oefinger, Sebastian Schmidt, and Johannes Betz. Imagined rollouts are kinematic, not dynamic: A diagnosis of long-horizon world-model failure. arXiv preprint arXiv:2607.05966, 2026. URL <https://arxiv.org/abs/2607.05966>.
- Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=F07ic7huE3>.
- Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pp. 43905–43941, 2025. doi: 10.52202/085713-1465.
- Ruixiang Sun, Hongyu Zang, Xin Li, and Riashat Islam. Learning latent dynamic robust representations for world models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 47234–47260. PMLR, 2024. URL <https://proceedings.mlr.press/v235/sun24n.html>.
- Yunhao Tang, Zhaohan Daniel Guo, Pierre Harvey Richemond, Bernardo Avila Pires, Yash Chandak, Rémi Munos, Mark Rowland, Mohammad Gheshlaghi Azar, Charline Le Lan, Clare Lyle, András György, Shantanu Thakoor, Will Dabney, Bilal Piot, Daniele Calandriello, and Michal Valko. Understanding self-predictive learning for reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 33632–33656. PMLR, 2023. URL <https://proceedings.mlr.press/v202/tang23d.html>.
- Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. arXiv preprint arXiv:2602.18639, 2026. URL <https://arxiv.org/abs/2602.18639>.
- Donna Vakalis. Operator-on-F complements value-equivalence: A planning-time diagnostic for latent world models. arXiv preprint arXiv:2607.04464, 2026. URL <https://arxiv.org/abs/2607.04464>.
- Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint-embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pp. 21897–21937, 2025. doi: 10.52202/085713-0740.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008. URL <https://www.jmlr.org/papers/v9/vandermaaten08a.html>.
- Tongzhou Wang, Simon S. Du, Antonio Torralba, Phillip Isola, Amy Zhang, and Yuandong Tian. Denoised MDPs: Learning world models better than the world itself. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 22591–22612. PMLR, 2022. URL <https://proceedings.mlr.press/v162/wang22c.html>.
- Ran Wei, Nathan Lambert, Anthony D. McDonald, Alfredo Garcia, and Roberto Calandra. A unified view on solving objective mismatch in model-based reinforcement learning. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=tQVZgvXhZb>.
- Fan Wu, Linyi Li, Zijian Huang, Yevgeniy Vorobeychik, Ding Zhao, and Bo Li. CROP: Certifying robust policies for reinforcement learning through functional smoothing. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022. URL <https://openreview.net/forum?id=HOjLHr1Zhmx>.

- Haiyu Wu, Randall Balestriero, and Morgan Levine. VISReg: Variance-invariance-sketching regularization for JEPA training. arXiv preprint arXiv:2606.02572, 2026. URL <https://arxiv.org/abs/2606.02572>.
- Han Yan, Zishang Xiang, Zeyu Zhang, and Hao Tang. MWM: Mobile world models for action-conditioned consistent prediction. arXiv preprint arXiv:2603.07799, 2026. URL <https://arxiv.org/abs/2603.07799>.
- Denis Yarats, Ilya Kostrikov, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=GY6-6sTvGaf>.
- Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=_SJ-_yyes8.
- Jewon Yeom, Hanseul Kim, Jeongjae Park, Sungmok Jung, Jaemin Lee, and Taesup Kim. What makes video world model latents action-relevant: Prediction over reconstruction. arXiv preprint arXiv:2606.07687, 2026. URL <https://arxiv.org/abs/2606.07687>.
- Hanzhe You, Yonggang Zhang, Maohao Ran, Zhiqin Yang, Zhenyuan Zhang, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. A control theory of predictability in latent world models. arXiv preprint arXiv:2607.10362, 2026. URL <https://arxiv.org/abs/2607.10362>.
- Yang Yu, Shiyuan Zhang, Yifei Sheng, Haoxiang Ren, and Haoxin Lin. How should world models be evaluated for embodied decision-making? a decision-making-centric position. arXiv preprint arXiv:2606.15032, 2026. URL <https://arxiv.org/abs/2606.15032>.
- Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=-2FCwDKRREu>.
- Junjian Zhang, Hao Tan, Ruonan Li, Dong Zhu, Aiping Li, and Zhaoquan Gu. ARB4WM: An adversarial robustness benchmark for world models in continuous control. arXiv preprint arXiv:2606.16605, 2026. URL <https://arxiv.org/abs/2606.16605>.

A PROOFS AND FIXED-POOL ANALYSIS

Proof of Proposition 1. Let $u = \bar{G}_a(E_\theta(h))$, $v = \bar{G}_a(E_\theta(\tilde{h}))$, and $y = Y_a^H$. Both errors use the same target y in the weighted rollout space of Equation (3). The reverse triangle inequality gives

$$|e_{\tilde{h}} - e_h| = | \|v - y\|_2 - \|u - y\|_2 | \quad (11)$$

$$\leq \|(v - y) - (u - y)\|_2 = \|v - u\|_2 = \text{ACPC}_H(h, \tilde{h}, \mathbf{a}). \quad (12)$$

The one-sided error increase satisfies the same bound because $(e_{\tilde{h}} - e_h)_+ \leq |e_{\tilde{h}} - e_h|$.

The following result uses the cost-change bounds for individual candidates to determine whether the selected candidate or elite set can change.

Proposition 3 (Fixed-pool selection certificates). *In the setting of Proposition 2, let w be the unique winner for the clean input. If its cost advantage over every other candidate exceeds the combined cost-change bounds,*

$$\min_{j \neq w} (C_j - C_w - b_j - b_w) > 0, \quad (13)$$

then it remains the winner for the perturbed input. Similarly, let $\mathcal{E}$ be the clean top- k elite set. If

$$\min_{i \in \mathcal{E}, j \notin \mathcal{E}} (C_j - C_i - b_j - b_i) > 0, \quad (14)$$

then the perturbed input produces the same elite set.

Corollary 1 (Conditional CEM stability). *Consider clean and perturbed CEM runs that start from the same proposal and use the same random samples. Suppose that, at a given iteration, they evaluate corresponding candidate action sequences. If Equation (14) holds, they select the same elite candidates and fit the same proposal for the next iteration. If the condition holds at every iteration, the two runs remain aligned. Because the evaluated solver returns the final proposal mean as its action, the selected actions are also identical.*

Proof of Proposition 2. For a single candidate, omit the index. The difference between the two squared costs factors as

$$|\tilde{C} - C| = \left| \|\tilde{x} - g\|_2^2 - \|x - g\|_2^2 \right| \quad (15)$$

$$= |\langle \tilde{x} - x, \tilde{x} + x - 2g \rangle| \quad (16)$$

$$\leq \|\tilde{x} - x\|_2 \|\tilde{x} + x - 2g\|_2 \quad (17)$$

$$\leq r(\|x - g\|_2 + \|\tilde{x} - g\|_2) = b. \quad (18)$$

The first inequality follows from Cauchy–Schwarz, and the second follows from the triangle inequality.

Let w and $\tilde{w}$ be the winners for the clean and perturbed inputs. Since $\tilde{w}$ minimizes the perturbed cost,

$$C_{\tilde{w}} \leq \tilde{C}_{\tilde{w}} + b_{\tilde{w}} \leq \tilde{C}_w + b_{\tilde{w}} \leq C_w + b_w + b_{\tilde{w}},$$

Since w minimizes the clean cost, $C_{\tilde{w}} - C_w \geq 0$. Together, these inequalities prove Equation (6).

For any clean winner w and competitor j ,

$$\tilde{C}_j - \tilde{C}_w \geq C_j - C_w - |\tilde{C}_j - C_j| - |\tilde{C}_w - C_w| \geq C_j - C_w - b_j - b_w.$$

Therefore, Equation (13) ensures that every competitor remains more costly than w . Applying the same argument to every $i \in \mathcal{E}$ and $j \notin \mathcal{E}$ proves that Equation (14) preserves the elite set. The strict inequalities exclude ties and make the result independent of the tie-breaking rule.

Proof of Corollary 1. Index the clean and perturbed runs by $b \in \{c, p\}$ and the CEM iterations by t . Let ϕ_t^b denote the proposal mean and variance for run b . Given shared random samples ω_t , the deterministic sampling map T produces the candidate pool

$$\mathcal{A}_t^b = T(\phi_t^b, \omega_t).$$

Each run selects the indices $\mathcal{E}_t^b$ of its k lowest-cost candidates. A deterministic, permutation-invariant update U then fits the next proposal:

$$\phi_{t+1}^b = U(\{\mathbf{a}^j : j \in \mathcal{E}_t^b\}).$$

The evaluated solver satisfies this condition because it uses the unweighted mean and standard deviation of the elite candidates.

The two runs start from the same proposal, so $\phi_0^c = \phi_0^p$. If their proposals agree at iteration t , the shared samples generate the same candidate pool. When Equation (14) holds, the two runs also select the same elite candidates and therefore fit the same next proposal:

$$\phi_t^c = \phi_t^p \implies \mathcal{A}_t^c = \mathcal{A}_t^p \implies \mathcal{E}_t^c = \mathcal{E}_t^p \implies \phi_{t+1}^c = \phi_{t+1}^p.$$

Thus, if the condition holds at every iteration, the proposals and candidate pools remain identical. The evaluated solver returns the final proposal mean, so the selected actions are also identical.

If a solver instead returns the best candidate from the final pool, Equation (13) must also hold at the last iteration. If either certificate cannot be verified, the proof no longer guarantees that the subsequent elite sets, proposals, or candidate pools agree.

Relation to ACPC. The planning-cost bound uses the final-step displacement r_j , whereas ACPC combines displacements across all H rollout steps. From Equation (3),

$$\sqrt{\alpha_H} r_j \leq \text{ACPC}_H \quad \text{when } \alpha_H > 0.$$

Thus, $r_j \leq \text{ACPC}_H / \sqrt{\alpha_H}$. In the planner experiments, we compute r_j directly instead of using this upper bound. This gives a tighter candidate-level measurement and does not require the observed future, but it requires rolling out each candidate from both the clean and perturbed histories.

Tightness of the bounds. Both bounds can hold with equality, so their right-hand sides cannot be reduced without additional assumptions. For the prediction-error bound, let u be a unit vector, set the common target to $Y = 0$, and let the two predicted rollouts be su and tu , where $s, t \geq 0$. Then

$$\|tu\|_2 - \|su\|_2 = \|tu - su\|_2,$$

so Equation (4) holds with equality. For the planning-cost bound, set $g = 0$, $x = su$, and $\tilde{x} = tu$. Then

$$|\tilde{C} - C| = |t - s|(t + s) = r(\|x - g\|_2 + \|\tilde{x} - g\|_2),$$

so the candidate cost-change bound is also attained exactly.

B LOCAL-GEOMETRY CASE-STUDY DEFINITIONS

This supplementary case study visualizes how augmentation contracts perturbed views relative to nearby clean histories. We use 128 logged PushT histories. For each history, we generate one clean view and 18 perturbed views, with six draws at each standard deviation in $\{0.01, 0.04, 0.08\}$. Each of the four qualitative t-SNE panels uses a separate deterministic projection (van der Maaten & Hinton, 2008), so coordinates are not compared across panels.

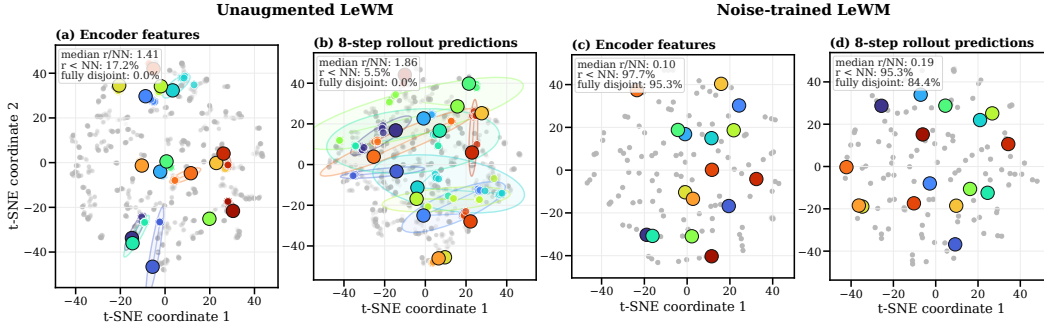


Figure 6: Local geometry for matched unaugmented (left) and $\sigma_{\max} = 0.08$ PushT checkpoints (right), at the encoder and after eight rollout steps. Markers show clean histories and perturbed views; summaries are computed in the original 192-dimensional space, not in t-SNE.

Without augmentation, median r/NN is 1.41 at the encoder and 1.86 after rollout, with no fully disjoint perturbation clouds. With augmentation, the ratios fall to 0.10 and 0.19, and the disjoint fractions rise to 95.3% and 84.4%. This is descriptive geometry, not evidence of task-semantic separation or planning robustness.

We evaluate two representation spaces: the encoder output, denoted by $\mathcal{V} = \text{enc}$, and the representation after eight autoregressive rollout steps, denoted by $\mathcal{V} = \text{roll}$. The rollout horizon is $H = 8$ (Section 3.1). For history i , let $v_{i,\mathcal{V}}^{(0)}$ be its clean representation and let $v_{i,\mathcal{V}}^{(m)}$, $m = 1, \dots, M$ with $M = 18$, be its perturbed representations. In the rollout space, the clean and perturbed views of the same history use the same recorded action sequence. We then define the sampled visual radius, the distance to the nearest other clean history, and their ratio:

$$\begin{aligned} r_{i,\mathcal{V}}^{\text{vis}} &= \max_{1 \leq m \leq M} \|v_{i,\mathcal{V}}^{(m)} - v_{i,\mathcal{V}}^{(0)}\|_2, \\ d_{i,\mathcal{V}}^{\text{NN}} &= \min_{j \neq i} \|v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)}\|_2, \quad \rho_{i,\mathcal{V}}^{\text{local}} = \frac{r_{i,\mathcal{V}}^{\text{vis}}}{d_{i,\mathcal{V}}^{\text{NN}}}. \end{aligned} \quad (19)$$

The figures abbreviate $\rho_{i,\mathcal{V}}^{\text{local}}$ as r/NN . A value below one means that every sampled perturbation moves the representation by less than the distance from its clean anchor to any other clean anchor. A value of at least one means that at least one perturbation displacement is as large as the nearest-clean distance. This comparison uses only distance magnitudes: it does not show that a perturbed representation approaches or overlaps another history, and the nearest clean history need not differ across a task-relevant state boundary.

The r/NN ratio considers only the perturbation radius of anchor i . To account for the perturbation radii of both histories, we also compare their enclosing balls. The ball around anchor i is *fully disjoint* if it does not intersect the ball around any other anchor:

$$\|v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)}\|_2 > r_{i,\mathcal{V}}^{\text{vis}} + r_{j,\mathcal{V}}^{\text{vis}} \quad \text{for every } j \neq i. \quad (20)$$

Across the 128 anchors, we report three summaries: the median r/NN ratio, the fraction with $r_{i,\mathcal{V}}^{\text{vis}} < d_{i,\mathcal{V}}^{\text{NN}}$, and the fraction satisfying the fully disjoint condition in Equation (20). The second summary compares each perturbation radius with the distance to the nearest other clean anchor. The third also accounts for the perturbation radius around every other anchor. All three summaries are computed in the original high-dimensional representation; overlap between the qualitative t-SNE ellipses is not used.

These summaries are descriptive and depend on the sampled histories and perturbation draws. They compare the encoder output and the final rollout step but do not examine intermediate predictions. They also do not bound changes in prediction error or planning cost; those theoretical links are provided by pairwise ACPC in Section 3.1.

C EVALUATION AND DIAGNOSTIC PROTOCOL

This appendix specifies the fixed evaluation and diagnostic protocol used in the main tables.

Evaluation grid. We evaluate LeWM on PushT, TwoRoom, Reacher, and Cube. Each checkpoint is evaluated with seeds 42, 43, and 44, using 100 episodes per seed. The primary evaluation adds Gaussian noise with standard deviation 0.08 to the history observations while keeping the goal image clean. For each task, the training grid contains one unaugmented checkpoint and eight checkpoints trained with full-sequence Gaussian-noise augmentation at $\sigma_{\text{max}} \in \{0.01, 0.02, \dots, 0.08\}$.

PLDM protocol. PLDM is evaluated on the same four tasks and nine-checkpoint training grid as LeWM, using the same evaluation seeds and number of logged histories. Its diagnostics use the same clean-perturbed pairing, eight-step rollout horizon, and SR definition. In the PLDM plots, raw IR is divided by the value of the unaugmented PLDM checkpoint from the same task and training run.

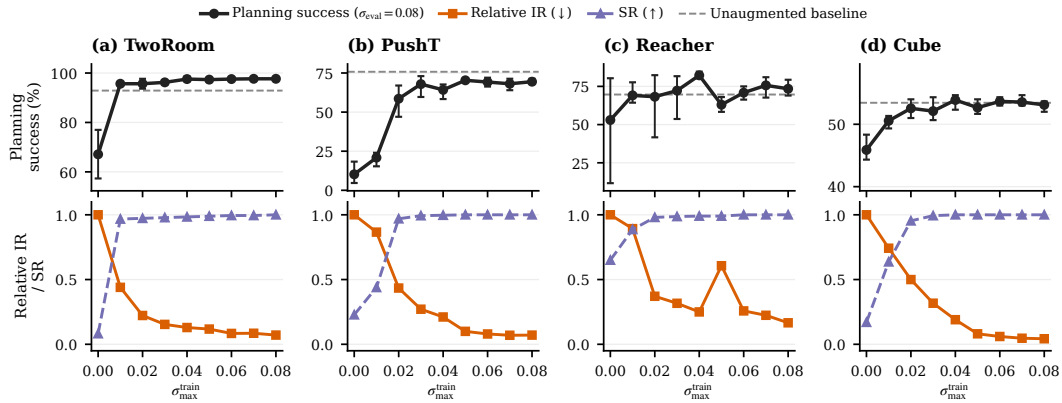


Figure 7: Complete PLDM Gaussian-noise training sweep over three runs. Curves show three-run means and success bars span the across-run range. The main-text summary includes every augmented level rather than selecting a recovery subset.

Success-rate criterion. For each task and training run, let P_{base} be the noisy-observation success rate of the unaugmented checkpoint, and let P_{upper} be the highest noisy-observation success rate observed in the fixed nine-checkpoint sweep. A checkpoint meets the success-rate criterion if its noisy-observation success rate is at least

$$P_{\text{base}} + 0.8(P_{\text{upper}} - P_{\text{base}})$$

and its clean-observation success rate is no more than five percentage points below that of the unaugmented checkpoint. At the task level, we call an augmentation level recovered only when this criterion holds in at least two of the three training runs. P_{upper} is used only to scale this recovery label; every level meeting the criterion remains part of the reported range.

IR protocol. We compute IR separately for each task, training run, and checkpoint. To express rollout divergence relative to the normal motion in each history, we first compute a clean-motion scale. For anchor i , let $u_{i,0}^{\text{obs}}$ be the projected embedding of the final history observation, and let $u_{i,1:H}^{\text{obs}}$ be the projected embeddings of the next H observed clean frames. We define

$$s_i = Q_{0.50} \left(\left\{ \|u_{i,k}^{\text{obs}} - u_{i,k-1}^{\text{obs}}\|_2 \right\}_{k=1}^H \right). \quad (21)$$

Thus, s_i is the median displacement between consecutive clean observations, including the transition from the final history observation to the first future observation.

For each anchor, we generate multiple perturbation draws using the visual shift being evaluated. The Gaussian-noise experiments use $\sigma = 0.08$; the blur and resize experiments apply their corresponding shifts. For every draw, we compute eight-step ACPC with uniform weights $\alpha_k = 1/H$ and normalize it by $s_i + \varepsilon_{\text{norm}}$. We average the normalized values across draws for each anchor and then take q90 across anchors. The result is raw IR in Equation (8).

SR compares different-state distances with raw IR from the same checkpoint. For the sweep plots, raw IR is divided by the value of the corresponding unaugmented checkpoint, as defined in Equation (10). Only the LeWM cross-task analysis uses relative IR for threshold selection. We use $\varepsilon_{\text{norm}} = 10^{-8}$ in Equation (7). Table 2 reports the results for the tested rollout horizons and summary quantiles.

Table 2: Horizon and quantile sensitivity of the IR reduction at fixed evaluation noise $\sigma = 0.08$. Each entry is the median, over the three training runs, of the ratio between the IR of the checkpoint trained at the highest augmentation level ($\sigma_{\text{max}} = 0.08$) and that of the unaugmented checkpoint; lower means a larger reduction. These values are descriptive and do not retune any threshold.

Task	$q = 0.90$ by horizon				$H = 8$ by quantile		
	H1	H2	H4	H8	q80	q90	q95
TwoRoom	0.05	0.04	0.05	0.06	0.06	0.06	0.06
PushT	0.06	0.07	0.06	0.09	0.07	0.09	0.09
Reacher	0.02	0.03	0.03	0.02	0.02	0.02	0.02
Cube	0.04	0.03	0.04	0.04	0.03	0.04	0.04

SR protocol. SR follows Equation (9). We first define a local neighborhood using the standardized logged endpoint states. Let $d_{0.35}$ be the 35th percentile of all pairwise distances between different endpoints—that is, the q35 of all off-diagonal Euclidean distances. This cutoff keeps the comparisons local while retaining an eligible neighbor for most anchors; Table 3 reports the resulting counts.

For each anchor i , we select the nearest history $j(i)$ that has a different endpoint-state label and lies within distance $d_{0.35}$. If no such history exists, the anchor is excluded from SR. We roll both histories forward under the anchor’s recorded action sequence $\mathbf{a}_i$. This sequence was not generally executed by the neighboring history, but using it for both rollouts prevents action differences from affecting the comparison. Their normalized rollout distance is

$$D_i^{\text{diff}} = \frac{\|\bar{G}_{\mathbf{a}_i}(E_\theta(h_i)) - \bar{G}_{\mathbf{a}_i}(E_\theta(h_{j(i)}))\|_2}{s_i + \varepsilon_{\text{norm}}}. \quad (22)$$

The pair counts as separated when D_i^{diff} is greater than raw q90 IR plus the fixed margin 0.10.

For pairing, the endpoint state is taken after the eight observed future steps and therefore records the state reached under that trajectory’s own actions. The dataset preprocessor standardizes each state coordinate, and neighborhood distances use the full standardized state vector. Endpoint-state labels are constructed by median splits of the task-specific coordinates listed in Table 3, using the fixed batch of 100 endpoints generated with anchor seed 9101. Because the logged endpoints, labels, and

neighborhoods are fixed before checkpoint evaluation, the selected pairs and eligible-anchor counts are identical across checkpoints and training runs.

Table 3: Endpoint-state labels used to construct SR pairs. The state is taken at the eighth observed future step, and its coordinates are standardized using full-dataset statistics. Counts report eligible and skipped histories among the fixed 100 anchors. These labels define operational partitions for the diagnostic; they are not semantic or action-relevance annotations.

Task	Logged state field (dim.)	Label construction $\ell(y)$	Eligible / skipped
TwoRoom	<code>pos_agent</code> (2)	Median split of the standardized agent x coordinate (<code>pos_agent[0:1]</code>).	61 / 39
PushT	<code>state</code> (7)	Three-bit label from median splits of standardized block x , block y , and block angle (<code>state[2:5]</code>).	98 / 2
Reacher	<code>observation</code> (6)	Three-bit label from median splits of the two standardized joint velocities and their Euclidean norm (<code>observation[4:6]</code>).	100 / 0
Cube	<code>observation</code> (28)	Three-bit label from median splits of the first two standardized coordinates and $\ r\ _2$, where $r = \bar{o}_{0:3} - \bar{o}_{25:28}$.	100 / 0

Threshold grids. For the LeWM cross-task evaluation, we search

$$t_{\text{IR}} \in \{0.05, 0.075, 0.10, 0.15, 0.20, 0.30\}, \quad t_{\text{SR}} \in \{0.80, 0.85, 0.90, 0.95\}.$$

For each nonempty subset of tasks used for threshold selection, we evaluate every threshold pair. We first minimize the mean number of augmentation-grid steps between the first checkpoint accepted by the IR–SR screen and the first checkpoint that meets the success-rate criterion. Remaining ties are resolved by, in order, fewer early acceptances, fewer late acceptances, higher balanced accuracy, a smaller t_{IR} , and a larger t_{SR} .

The selected thresholds are then applied unchanged to the tasks that were not used for their selection. Success-rate labels from those tasks and all blur or resize results are excluded from threshold selection.

Reproducibility. The commands used to rebuild the paper figures and tables are documented in `paper1/scripts/README.md` and `tools/README_paper1.md`. The released machine-readable summaries are sufficient for the reader-facing aggregation and plotting steps. Recomputing checkpoint-level diagnostics additionally requires the released checkpoints and datasets. The summaries record the training seeds, evaluation seeds, and protocol hashes used for each result.

Table 4: Recovery ranges in the original Gaussian-noise training sweep (nine checkpoints per task: no augmentation plus eight noise levels; Figure 2 shows every level). The final column lists the contiguous training-noise ranges that meet the prespecified success-rate criterion in Section 4.1. It summarizes an accepted range rather than selecting a single training strength.

Task	Unaugmented success (%)	Training levels meeting criterion
TwoRoom	68.8	0.01–0.08
PushT	7.2	0.03–0.08
Reacher	18.2	0.03–0.08
Cube	43.1	0.03–0.07

D ADDITIONAL MULTI-SEVERITY ANALYSIS

This section gives the aggregation and sensitivity details behind Section 4.3. The broad-severity experiment is analyzed separately from the original narrow sweep. It keeps the original thresholds and pair construction fixed and uses continuous retention rather than defining a new performance band.

The same 3 TwoRoom checkpoints account for every additional SR rejection among IR-passing rows at $\sigma_{\text{eval}} = 0.08$ and 0.16: 1 has $\sigma_{\text{max}} = 1.0$ and 2 have $\sigma_{\text{max}} = 1.5$. At $\sigma_{\text{eval}} = 0.32$, these rejections

comprise 4 TwoRoom, 5 PushT, and 1 Reacher checkpoints; at 0.64, they comprise 5 TwoRoom and 6 PushT checkpoints. No Cube checkpoint passes IR while failing SR. The conditional rejections thus span three tasks at 0.32 and two at 0.64, although some checkpoint rows recur across severity slices and the conditional denominator also changes. All low-severity rejections fall on TwoRoom, whose SR catalog uses one median split and has the fewest eligible anchors (61 of 100; Table 3); this concentration should be read within the scope of the tested pair catalog.

Table 5: Fixed-threshold decision decomposition in the separate broad-severity experiment. Each row partitions the same 96 trained checkpoints at one evaluation severity. “IR pass, SR fail” is the additional rejection shown in Figure 3. The reverse disagreement appears at the two higher severities, so neither condition subsumes the other.

σ_{eval}	Pass both	IR pass, SR fail	IR fail, SR pass	Fail both
0.08	93	3	0	0
0.16	93	3	0	0
0.32	72	10	5	9
0.64	35	11	5	45

The reverse disagreement in Table 5 is expected to be possible because the conditions use different references. Relative IR divides the raw perturbation radius by the unaugmented checkpoint’s radius at the same severity. SR instead compares that raw radius with clean different-state distances from the checkpoint under evaluation. The fixed-threshold results therefore cannot be reproduced by nesting one condition inside the other.

For each task and training run, the grid contains eight training-noise maxima and four evaluation severities. Let x_{ij} denote any diagnostic or behavioral quantity at training level i and evaluation severity j . We remove the two additive main effects using

$$\tilde{x}_{ij} = x_{ij} - \bar{x}_{i.} - \bar{x}_{.j} + \bar{x}_{..}$$

Spearman correlations are computed over the 32 centered cells within a single task and training run. We then average the 3 training-run correlations within each task and weight the 4 tasks equally. The task ranges in Table 6 summarize heterogeneity; they are not confidence intervals. Double-centering leaves $(8 - 1)(4 - 1) = 21$ interaction degrees of freedom and imposes row- and column-sum constraints, so the 32 cells are not treated as independent observations.

Table 6: Descriptive Spearman associations with same-checkpoint retention (success under noise minus clean success) in the separate broad-severity experiment. Within each task and training run, all quantities are double-centered over the 8×4 training–evaluation severity grid. Training-run correlations are then averaged within each task, and tasks receive equal weight. The adjusted column reports partial Spearman correlation after residualizing the rank-transformed diagnostic and outcome against the rank-transformed exposure ratio $-\sigma_{\text{eval}}/\sigma_{\text{max}}$ within each task–run block. Ranges contain the four task means, not confidence intervals. The exposure ratio is a simple metadata comparator, not an exhaustive baseline, and these associations do not establish checkpoint-selection accuracy.

Quantity (favorable direction)	Direct ρ	Task range	Adjusted ρ	Task range
$-\sigma_{\text{eval}}/\sigma_{\text{max}}$	0.787	[0.662, 0.887]	–	–
$-\text{IR}^{\text{rel}}$	0.796	[0.671, 0.909]	0.458	[0.070, 0.667]
SR	0.826	[0.756, 0.900]	0.532	[0.225, 0.694]

On PushT, the exposure comparator’s direct task-level correlation exceeds both diagnostics. This is why we use it as an explicit design-metadata comparator and do not interpret the small differences among the direct aggregate values as diagnostic superiority.

Using stress success divided by same-checkpoint clean success instead of their percentage-point difference gives the same qualitative result. The equal-task mean direct correlations are 0.813 for lower relative IR and 0.839 for higher SR; after accounting for the exposure ratio they are 0.560 and 0.626, respectively. Every clean-success estimate is nonzero (minimum 7%), but ratios can still amplify changes for weak checkpoints; we therefore keep the percentage-point difference as the primary outcome and treat the ratio only as a sensitivity check. These are descriptive factorial

associations. The 384 rows are not independent replicates, and the four evaluation-severity rows for a checkpoint share the same clean-performance estimate; therefore we do not attach row-level p -values or confidence intervals.

E PREDICTION-ERROR SCOPE AND CONTROLS

We checked Proposition 1 for every logged eight-step pair and every candidate-specific five-step pair across all tasks, training runs, and training conditions. No numerical violations occurred. This check confirms that the implementation follows the inequality. It is not an independent statistical success count or evidence of model quality, because the inequality holds for every correctly computed pair.

The prediction-error and planning experiments answer different questions. The prediction-error experiment measures how a visual perturbation changes rollout error relative to the observed future under the recorded actions. The planning experiment instead computes ACPC for the action candidates evaluated by CEM and relates it to selection regret. We therefore treat the planning analysis as separate evidence rather than infer it from the prediction-error result (Section 4.5).

The prediction-error analysis uses only the unaugmented checkpoints and Gaussian-noise levels $\sigma \in \{0.02, 0.05, 0.08\}$. Table 7 reports the relative reduction in cross-validated MAE for each training run. Table 8 reports the corresponding MAE values and the selected destroyed-action controls. Base+ACPC₈ achieves lower MAE than both comparison models on all four tasks in every training run.

Table 7: Relative reduction in out-of-group MAE from Base+ACPC₈ when predicting the error drift d on the unaugmented checkpoints (protocol and model definitions in Section 4.4). Base+Control₈ uses the per-cell oracle control; higher is better. The last column counts task–run cells in which Base+ACPC₈ is best.

Training run (seed)	vs. Base	vs. Base+Control ₈	Task–run cells improved
3072	55.5%	51.4%	4/4
3073	60.8%	54.8%	4/4
3074	51.4%	47.7%	4/4
mean $\pm$ SD	55.9 $\pm$ 4.7%	51.3 $\pm$ 3.5%	12/12

Table 8: Out-of-group MAE (16-fold leave-one-trajectory-group-out) for predicting the error drift d on the unaugmented checkpoints; lower is better, model definitions in Section 4.4. “Selected control” is the destroyed-action control chosen by the per-cell oracle; “paired wins” counts the folds (of 16) in which Base+ACPC₈ beats Base+Control₈.

Task	run (seed)	Base	Base +Control ₈	Base +ACPC ₈	selected control	paired wins
TwoRoom	3072	2.535	2.099	0.755	shuffled actions	15/16
TwoRoom	3073	2.336	2.015	0.866	shuffled actions	15/16
TwoRoom	3074	2.075	1.911	1.006	swapped actions	13/16
PushT	3072	2.068	2.068	1.762	shuffled actions	11/16
PushT	3073	1.969	2.008	1.315	zeroed actions	13/16
PushT	3074	1.909	1.833	1.439	zeroed actions	13/16
Reacher	3072	1.358	1.097	0.288	shuffled actions	16/16
Reacher	3073	1.399	0.793	0.247	shuffled actions	16/16
Reacher	3074	1.409	1.136	0.263	shuffled actions	16/16
Cube	3072	1.171	1.037	0.489	zeroed actions	14/16
Cube	3073	1.416	1.212	0.500	zeroed actions	14/16
Cube	3074	1.061	1.008	0.552	shuffled actions	14/16

Table 9: Joint IR/SR checkpoint screening with thresholds chosen on other tasks. Thresholds are selected on the threshold-selection tasks and applied unchanged to all test tasks. Metrics first average training runs within each test task and then weight tasks equally. Recovery-onset mismatch is measured in training-augmentation grid levels (one level is 0.01 in $\sigma_{\max}$).

Selection tasks	Test tasks	t_{IR}	t_{SR}	BA	P / R	Onset mismatch mean / max
TwoRoom	PushT, Reacher, Cube	0.3	0.95	0.888	0.884 / 0.981	0.3 / 1.0
PushT	TwoRoom, Reacher, Cube	0.3	0.95	0.894	0.902 / 0.956	0.6 / 1.0
Reacher	TwoRoom, PushT, Cube	0.1	0.95	0.723	0.906 / 0.540	2.9 / 5.0
Cube	TwoRoom, PushT, Reacher	0.3	0.95	0.913	0.950 / 0.938	0.6 / 1.0
TwoRoom, PushT	Reacher, Cube	0.3	0.95	0.874	0.853 / 1.000	0.3 / 1.0
TwoRoom, Reacher	PushT, Cube	0.3	0.95	0.887	0.873 / 0.972	0.3 / 1.0
TwoRoom, Cube	PushT, Reacher	0.3	0.95	0.903	0.925 / 0.972	0.3 / 1.0
PushT, Reacher	TwoRoom, Cube	0.3	0.95	0.896	0.901 / 0.935	0.7 / 1.0
PushT, Cube	TwoRoom, Reacher	0.3	0.95	0.912	0.952 / 0.935	0.7 / 1.0
Reacher, Cube	TwoRoom, PushT	0.3	0.95	0.926	0.972 / 0.907	0.7 / 1.0
TwoRoom, PushT, Reacher	Cube	0.3	0.95	0.858	0.802 / 1.000	0.3 / 1.0
TwoRoom, PushT, Cube	Reacher	0.3	0.95	0.889	0.905 / 1.000	0.3 / 1.0
TwoRoom, Reacher, Cube	PushT	0.3	0.95	0.917	0.944 / 0.944	0.3 / 1.0
PushT, Reacher, Cube	TwoRoom	0.3	0.95	0.935	1.000 / 0.869	1.0 / 1.0

F CROSS-TASK THRESHOLD PARTITIONS

With four tasks, there are 14 nonempty choices of tasks for threshold selection: four single-task choices, six two-task choices, and four three-task choices. The selected thresholds are applied to the remaining task or tasks. Each evaluation includes all three training runs and all nine checkpoints for every remaining task. Checkpoints are not counted as independent samples: metrics first average the training runs within each evaluation task and then weight the evaluation tasks equally.

When thresholds are selected using only Reacher, the procedure chooses $(t_{\text{IR}}, t_{\text{SR}}) = (0.1, 0.95)$ instead of the predominant $(0.3, 0.95)$. Both threshold pairs locate Reacher recovery within one augmentation-grid step, but $t_{\text{IR}} = 0.1$ matches the Reacher onset exactly and therefore wins the tie-break. Recovered Reacher checkpoints have relative IR well below 0.1, whereas the earliest recovered checkpoints on the other tasks remain above 0.1. The Reacher-only threshold therefore accepts recovery on those tasks too late, by as many as five augmentation levels.

G CROSS-STRESSOR PAIRS AND BOUNDARY CASES

This section reports IR and SR separately for the 24 matched blur/resize checkpoint pairs. Component-wise reporting avoids assigning a continuous joint score to diagnostics with different roles. A pair is labeled behaviorally positive when the augmented checkpoint improves success under the evaluated shift by at least five percentage points while losing no more than five clean-success points relative to its unaugmented counterpart.

Table 10: Descriptive cross-stressor component summary. IR improvement is $1 - \text{IR}_{\text{rel}}$ because the base value is 1; SR change is endpoint SR minus base SR. Spearman values correlate each component change with planning-success change. Binary agreement compares a positive component change with the prespecified positive behavior label. “IR pass / veto” reports absolute endpoint IR passes and the subset rejected by SR. The final cell gives the 2.5th–97.5th percentiles from exact enumeration of all $4^4 = 256$ ordered task-cluster resamples, using linear interpolation. This is a sensitivity distribution, not a new-task population CI; the 24 paired rows are not IID.

Scope	n	ρ_{IR}	ρ_{SR}	binary agreement	IR pass / veto	cluster-resampling
				IR / SR		sensitivity
Pooled	24	0.854	0.913	0.917 / 0.917	6 / 0	IR: [0.263, 0.950] SR: [0.352, 0.929]
Blur	12	0.874	0.953	0.917 / 0.917	3 / 0	–
Resize	12	0.748	0.846	0.917 / 0.917	3 / 0	–
TwoRoom	6	0.543	0.543	1.000 / 1.000	0 / 0	–
PushT	6	0.464	0.603	0.667 / 0.667	0 / 0	–
Reacher	6	0.886	0.886	1.000 / 1.000	6 / 0	–
Cube	6	-0.086	-0.543	1.000 / 1.000	0 / 0	–
LOTO range	18	[0.720, 0.911]	[0.846, 0.901]	–	–	–

The two label disagreements are PushT run 3072 under resize and run 3074 under blur. Each gains four success points, one below the prespecified five-point cutoff, while both diagnostic components move in the favorable direction.

Table 11: Diagnostic ablation on 24 paired blur/resize endpoints. Each metric predicts improvement when its endpoint-to-reference ratio is below one; no threshold is fitted or transferred. Spearman correlation relates diagnostic improvement, defined as one minus this ratio, to planning-success change. Correct/24 and BA compare the predicted sign with the prespecified behavioral label. The LOTO range recomputes pooled Spearman after omitting each task; the 24 pairs are clustered within four tasks.

Diagnostic	Spearman	Correct / 24	BA	LOTO Spearman range
Encoder only	0.859	20	0.778	[0.725, 0.901]
One-step rollout	0.738	20	0.778	[0.454, 0.877]
Zero actions	0.805	21	0.833	[0.626, 0.877]
Cross-anchor action shuffle	0.780	21	0.833	[0.586, 0.869]
Temporal action shuffle	0.844	22	0.889	[0.678, 0.923]
Recorded-action ACPC	0.854	22	0.889	[0.720, 0.911]

H LOCAL SENSITIVITY UNDER GAUSSIAN NOISE

To relate ACPC under Gaussian noise to local model sensitivity, consider a small isotropic perturbation $\delta x \sim \mathcal{N}(0, \sigma^2 I)$. Let J_E and J_G be the Jacobians of the encoder and rollout map, evaluated along the clean input. A first-order expansion gives

$$\Delta G = J_G J_E \delta x + o(\|\delta x\|_2).$$

Writing $A = J_G J_E$ and using $\mathbb{E}[\delta x \delta x^\top] = \sigma^2 I$, we obtain

$$\mathbb{E}[\|\Delta G\|_2^2] = \text{tr}(A \mathbb{E}[\delta x \delta x^\top] A^\top) = \sigma^2 \text{tr}(A A^\top) = \sigma^2 \|A\|_F^2, \quad (23)$$

Therefore, for small σ ,

$$\mathbb{E}[\|\Delta G\|_2^2] \approx \sigma^2 \|J_G J_E\|_F^2.$$

The squared Frobenius norm of the composed Jacobian thus measures how strongly the encoder–rollout path locally amplifies Gaussian observation noise. Jacobian Frobenius norms have previously been used to measure and regularize local representation sensitivity Rifai et al. (2011). We estimate

$$\|J_G J_E\|_F^2 = \text{tr}[(J_G J_E)^\top (J_G J_E)]$$

with Rademacher probes and the Hutchinson estimator Hutchinson (1989). Jacobian–vector products compute these estimates without constructing the full Jacobian. In every task, both the finite-difference

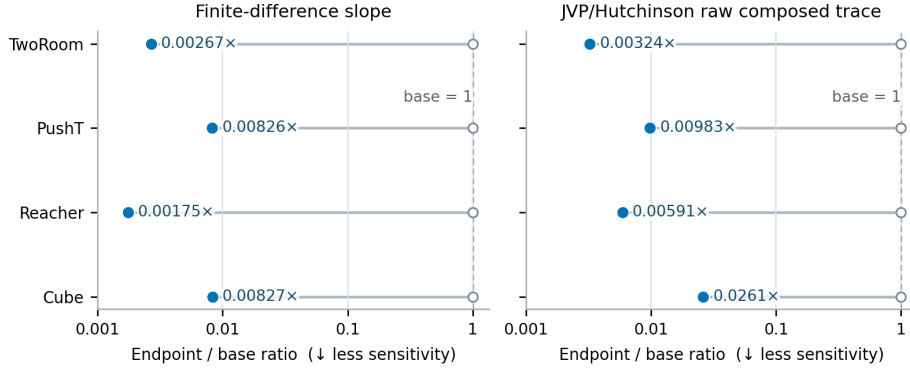


Figure 8: Ratio of local sensitivity for the Gaussian-noise-augmented checkpoint to that of the unaugmented checkpoint. We report a finite-difference estimate and a JVP-based Hutchinson estimate of the composed Jacobian’s squared Frobenius norm. Both estimates are below one for every task, indicating lower local sensitivity after augmentation. These local measurements do not establish task performance or global robustness.

and Jacobian-based estimates are lower for the noise-augmented checkpoint (Figure 8). This result is consistent with Gaussian-noise augmentation reducing ACPC by lowering the local sensitivity of the encoder-rollout path. The analysis applies only near the evaluated inputs and does not provide a global robustness guarantee.

I FIXED-POOL CERTIFICATES AND ADAPTIVE-CEM SELECTION REGRET

The selection-regret analysis does not use task-success labels. We evaluate the unaugmented and $\sigma_{\max} = 0.08$ checkpoints from every task and training run. For each checkpoint, we use 100 histories and apply perturbations at severities 0.02, 0.05, and 0.08. Each clean or perturbed CEM run uses 64 candidate action sequences, eight elite candidates, eight iterations, and a five-step prediction horizon. The paired runs start from the same proposal and use the same random samples, but each run updates its proposal using its own elite candidates.

The implementation averages the squared embedding error across the d embedding coordinates:

$$C_j = \frac{\|x_j - g\|_2^2}{d}.$$

This differs from the summed squared cost in Proposition 2 only by the positive factor $1/d$. The factor rescales every C_j and b_j equally, so the regret bound and selection certificates remain unchanged.

For the fixed-pool certificate analysis, we capture CEM’s ordered first-round proposal of 64 candidates for each history and evaluate that same pool from the clean and perturbed inputs. At each severity, we average the rates equally over 12 task–training-run blocks, with 100 histories per block. The same histories recur across severities and checkpoint roles, so the entries in Table 12 are paired descriptive rates rather than independent observations. The candidate-level cost bound is evaluated directly from the two predicted endpoints and the fixed goal, as in Equation (5).

Table 12: Empirical coverage of the candidate-specific planning-cost certificates on the fixed first-round CEM pool. Each entry is the equal-weight mean over 12 task-training-run blocks, with 100 histories and the same 64 candidates per clean-perturbed pair. “Top-1 cert.” and “Elite cert.” are the fractions satisfying Equation (13) and Equation (14), respectively; “Same top-1” is the observed fraction whose lowest-cost candidate is unchanged. The zero false certificates and zero cost-bound violations are deterministic implementation checks. These results concern one fixed candidate pool, not subsequent adaptive CEM iterations or simulator return.

Checkpoint	Probe σ	Top-1 cert. (%)	Elite cert. (%)	Same top-1 (%)
Unaugmented	0.02	2.3	0.0	67.4
Unaugmented	0.05	0.0	0.0	20.3
Unaugmented	0.08	0.0	0.0	8.7
Augmented ($\sigma_{\max} = 0.08$)	0.02	69.5	30.4	98.2
Augmented ($\sigma_{\max} = 0.08$)	0.05	46.5	14.0	96.3
Augmented ($\sigma_{\max} = 0.08$)	0.08	34.8	8.8	93.3

For each CEM run, we compute ACPC for every candidate under that candidate’s action sequence. We summarize the candidate pool by the q90 ACPC at horizons one and five. Let $\mathbf{a}$ be the plan selected from the clean history and $\tilde{\mathbf{a}}$ the plan selected from its perturbed counterpart. The regression target is the extra predicted cost of the perturbed-history plan when both plans are evaluated from the clean history:

$$[C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a})]_+.$$

Within each training run, we fit ridge regressions on three tasks and evaluate them on the remaining task, rotating through all four choices. MAE is computed on $\log(1 + \text{selection regret})$ and averaged equally across the four evaluation tasks. The relative MAE reduction is computed separately for each training run before reporting the mean and sample standard deviation across the three runs. This experiment evaluates whether ACPC reflects the model-cost effect of a CEM selection change; it does not evaluate simulator return.